claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-ae5e931eaa6879494.jsonl`
- SHA-256: `7143e0613a03b6b3520e058fd76e0c4356f7f823d242ab90fb33051cdbd0f8f8`
- Source modified: `2026-06-13T19:16:49+00:00`
- Imported at: `2026-07-08T16:00:00+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:13:53.195Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"NO-REGRESSION verified: wasb_hybrid / tracknet_hybrid are byte-identical","severity":"info","file":"backend/pipeline/segmenters/trajectory_hybrid.py:81-104, 200-265","detail":"I read both new files, both edited twins, the pre-change HEAD of trajectory_hybrid.py, add_candidate_segment, rally_gate.gate_windows, and ran the suite. Every twin-facing claim holds:\n\n(1) PERSISTED STATS UNCHANGED. The base _enrich_candidate_stats (lines 89-94) returns exactly the 4 motion keys {peak_velocity, mean_velocity, active_frame_count, track_id_count} in the same insertion order as the old inline `stats` dict (HEAD:180-184). add_candidate_segment serializes via json.dumps(stats) (database.py:255), and window_stats[i] is passed unchanged (line 216) -> identical JSON row. Pinned by test_persisted_stats_are_exactly_the_four_motion_keys.\n\n(2) HANDOFF SENDS ALL WINDOWS. Base _select_for_handoff returns list(range(len(windows))) = [0..N-1] (line 104). New futures comprehension `for wi in handoff_indices` with that list yields the same (wi, all_candidate_windows[wi]) pairs and the same `for future in futures` result-collection order as the old `enumerate(all_candidate_windows)` (HEAD:227-228). _candidate_id linkage via candidate_ids[wi] is unaffected. Pinned by test_base_fusion_hooks_are_identity.\n\n(3) PHASE-3/PHASE-4 UNCHANGED. process_candidate_window body, padding math, provider call, timestamp offsetting, os.remove/os.rmdir, and the entire Phase-4 DB-population/link loop are untouched by the diff.\n\n(4) EMPTY-WINDOWS GUARD INTACT. `if not all_candidate_windows: return []` (lines 219-220) still sits after persistence and BEFORE _select_for_handoff (line 225), so on empty input window_stats=[], the persistence loop runs zero times, and the function returns [] before any gating call.\n\n(5) store_candidates=False CAUSES NO BEHAVIOURAL CHANGE for the twins. window_stats is now computed unconditionally (lines 203-206), but the base hook is pure (reads dict keys only, no I/O, no mutation), so computing-then-discarding it is dead compute with no observable effect.\n\nDefault FusionSegmenter also verified: _build_runner reads WasbConfig.from_indexing_cfg(idx_cfg) from idx_cfg['wasb'] (wasb_runner.py:46), i.e. the identical WASB runner to wasb_hybrid; default min_crossings=0/min_players=0/person OFF makes _select_for_handoff keep every index (fusion_hybrid.py:120-127), so it seeds and hands off identically to its substrate. Pinned by test_default_hands_off_all_windows + test_persisted_candidates_are_superset_of_substrate_windows. All 42 tests in the 4 relevant files pass."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py,

backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:13:56.165Z)

I'll verify this NO-REGRESSION finding by reading the actual code. Let me start by examining the relevant files and the diff.

3. user (2026-06-13T19:13:57.803Z)

```
warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be
replaced by CRLF the next time Git touches it
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
index b2847c9..149282e 100644
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
@@ -78,6 +78,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         except (ValueError, TypeError):
             return max(3, int(duration / 0.8))

+    # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -------
+    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
+                                frame_width: float, video_path: str,
+                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
+        """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
+        returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
+        rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
+        features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
+        return {
+            "peak_velocity": cw["peak_velocity"],
+            "mean_velocity": cw["mean_velocity"],
+            "active_frame_count": cw["active_frame_count"],
+            "track_id_count": cw["track_id_count"],
+        }
+
+    def _select_for_handoff(self, windows: List[Dict[str, Any]],
+                            window_stats: List[Dict[str, Any]],
+                            idx_cfg: Dict[str, Any]) -> List[int]:
+        """Indices of the candidate windows that proceed to the AI handoff (and thus may
+        become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
+        ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
+        Persisted candidates are NOT affected ? they remain the full superset for offline
+        re-scoring."""
+        return list(range(len(windows)))
+
     def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                       player_pool: Optional[List[str]] = None,
                       max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
@@ -172,31 +197,36 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                             f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                             f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

+        # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
+        # segmenter adds net_crossings + person-cue features. Computed once, reused for both
+        # persistence and the handoff gate below.
+        window_stats = [
+            self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
+            for cw in all_candidate_windows
+        ]
+
         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
```

```
         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
         if store_candidates:
             for i, cw in enumerate(all_candidate_windows):
-                stats = {
-                    "peak_velocity": cw["peak_velocity"],
-                    "mean_velocity": cw["mean_velocity"],
-                    "active_frame_count": cw["active_frame_count"],
-                    "track_id_count": cw["track_id_count"],
-                }
                 candidate_ids[i] = self.db.add_candidate_segment(
                     video_id, detector=self.name,
                     start_time=cw["start"], end_time=cw["end"],
                     chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
-                    stats=stats, detection_params=detection_params,
+                    stats=window_stats[i], detection_params=detection_params,
                 )

         if not all_candidate_windows:
             return []

         # --- Phase 3: AI provider handoff over padded candidate clips ---
+        # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
+        # candidates above stay the full superset ? only the served play_segments are gated.
+        handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
         os.makedirs(handoff_dir, exist_ok=True)
         logger.info("Phase 3: %s validation on %d candidate windows...",
-                    provider_name, len(all_candidate_windows))
+                    provider_name, len(handoff_indices))

         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
             w_start = max(0, window["start"] - handoff_padding)
@@ -225,8 +255,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):

         ai_segments: List[dict] = []
         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
-            futures = [executor.submit(process_candidate_window, wi, cw)
-                       for wi, cw in enumerate(all_candidate_windows)]
+            futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
+                       for wi in handoff_indices]
             for future in futures:
                 ai_segments.extend(future.result())
         try:
```

4. assistant (2026-06-13T19:14:00.491Z)

Now let me read the new files and the base hooks. Let me gather everything in parallel.

5. user (2026-06-13T19:14:01.199Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
```

```
    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
                          detection_params extras such as weights_path).

    This base is deliberately NOT registered; only concrete subclasses register.
    """
    import os
    import time
    import logging
    import concurrent.futures
    from typing import Any, Dict, List, Optional, Tuple

    import cv2

    from backend.pipeline.segmenters.base import BasePlaySegmenter
    from backend.utils.video import get_video_duration, extract_video_chunk
    from backend.sports import sport_registry
    from backend.providers import provider_registry
    from backend.pipeline.detectors.base import DetectorRunner

    logger = logging.getLogger(__name__)


    def _fmt_ts(seconds: float) -> str:
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    class TrajectoryHybridSegmenter(BasePlaySegmenter):
        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""

        #: config section under `indexing` (velocity_threshold lives there), the
        #: output subdir for trajectories, and the metadata detector-tag prefix.
        family: str = ""
        #: human-readable label used in log lines.
        display_label: str = ""

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
            """Return (runner, detector-specific detection_params extras)."""
            raise NotImplementedError

        @staticmethod
        def _probe_fps_width(video_path: str) -> tuple:
            """Return (fps, frame_width) for a video, with sensible fallbacks."""
            cap = cv2.VideoCapture(video_path)
            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
            cap.release()
            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

        @staticmethod
        def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
            """Provider-reported exchange count, else a duration-based estimate.

            One canonical implementation ? the twins had drifted (wasb relied on
            ``int(None)`` raising into the fallback; same result, by accident).
            """
            shot_count = segment.get("shuttle_exchange_count")
            if shot_count is None:
                return max(3, int(duration / 0.8))
            try:
                return int(shot_count)
```

```
78              except (ValueError, TypeError):
79                  return max(3, int(duration / 0.8))
80
81          # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
82          def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
83                                      frame_width: float, video_path: str,
84                                      idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
85              """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
86              returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
87              rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
88              features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
89              return {
90                  "peak_velocity": cw["peak_velocity"],
91                  "mean_velocity": cw["mean_velocity"],
92                  "active_frame_count": cw["active_frame_count"],
93                  "track_id_count": cw["track_id_count"],
94              }
95
96          def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                                  window_stats: List[Dict[str, Any]],
98                                  idx_cfg: Dict[str, Any]) -> List[int]:
99              """Indices of the candidate windows that proceed to the AI handoff (and thus may
100             become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101             ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102             Persisted candidates are NOT affected ? they remain the full superset for
offline
103             re-scoring."""
104             return list(range(len(windows)))
105
106         def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
107                           player_pool: Optional[List[str]] = None,
108                           max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109             # override-ignore: the ABC declares the Result contract (Success|Failure)
110             # but every live segmenter still returns a bare list ? the documented
111             # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112             label = self.display_label
113             # --- Sport profile + AI provider wiring (identical across segmenters) ---
114             sport_name = self.config.get("sport", "badminton")
115             try:
116                 sport_profile = sport_registry.get(sport_name)
117             except KeyError as e:
118                 logger.error(str(e))
119                 return []
120
121             idx_cfg = self.config.get("indexing", {})
122             provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125             api_key_env_var = f"{provider_name.upper()}_API_KEY"
126             api_key = os.environ.get(api_key_env_var)
127             if not api_key:
128                 logger.error(
129                     f"{api_key_env_var} environment variable is not set! "
130                     f"To run the {provider_name} handoff, set your API key. "
131                     f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
132                 )
133                 return []
134             try:
135                 provider = provider_registry.get(provider_name, api_key=api_key,
```

```
model_name=model_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

            # --- Runner config + windowing thresholds ---
            runner, runner_params = self._build_runner(idx_cfg)

            velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
            merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
            min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
            handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
            ai_max_workers = idx_cfg.get("ai_max_workers", 5)
            log_candidates = idx_cfg.get("log_yolo_candidates", True)
            store_candidates = idx_cfg.get("store_yolo_candidates", True)

            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s WSL environment not ready: %s", label, msg)
                return []
            logger.info("%s WSL env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
            t_start = time.time()
            out_dir = os.path.join("output", self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
                logger.error("%s produced no trajectory; aborting.", label)
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))

            all_candidate_windows = runner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width, chunk_start=0.0,
                velocity_thresh=velocity_thresh, merge_gap=merge_gap,
                min_window_duration=min_window_duration, chunk_index=0,
            )
            logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                        label, time.time() - t_start, len(all_candidate_windows))

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(f"[{label} rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                                f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
```

```
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector=self.name,
                        start_time=cw["start"], end_time=cw["end"],
                        chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
                        stats=window_stats[i], detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
gated.
            handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
            handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
            os.makedirs(handoff_dir, exist_ok=True)
            logger.info("Phase 3: %s validation on %d candidate windows...",
                        provider_name, len(handoff_indices))

            def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                w_start = max(0, window["start"] - handoff_padding)
                w_end = min(video_duration, window["end"] + handoff_padding)
                w_dur = w_end - w_start
                w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
                    return []
                prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
                                                     label=f"Handoff {wi+1}")
                valid = []
                for seg in segments:
                    try:
                        seg["start_time"] = float(seg["start_time"]) + w_start
                        seg["end_time"] = float(seg["end_time"]) + w_start
                        seg["_candidate_id"] = candidate_ids[wi]
                        valid.append(seg)
                    except (ValueError, TypeError, KeyError) as e:
                        logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
                try:
                    os.remove(w_path)
                except OSError:
```

```
253                     pass
254                 return valid
255
256             ai_segments: List[dict] = []
257             with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
258                 futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
259                            for wi in handoff_indices]
260                 for future in futures:
261                     ai_segments.extend(future.result())
262             try:
263                 os.rmdir(handoff_dir)
264             except OSError:
265                 pass
266             logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267
268             # --- Phase 4: DB population + candidate linking ---
269             segments_data = []
270             for segment in ai_segments:
271                 try:
272                     start = float(segment["start_time"])
273                     end = float(segment["end_time"])
274                     dur = end - start
275                     metadata = {
276                         "shot_count": self._shot_count_from(segment, dur),
277                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278                         "detector": f"{self.family}-hybrid-{model_name}",
279                     }
280                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281                     if segment_id:
282                         candidate_id = segment.get("_candidate_id")
283                         if candidate_id is not None:
284                             self.db.link_candidate_to_segment(candidate_id, segment_id)
285                         segments_data.append({
286                             "id": segment_id, "start_time": start, "end_time": end,
287                             "duration": dur, "metadata": metadata,
288                         })
289                 except Exception as e:
290                     logger.warning("Skipping invalid segment struct: %s", e)
291
292             return segments_data
293
```

6. assistant (2026-06-13T19:14:03.791Z)

Now let me verify claim (1) about the pre-change HEAD ordering, and read the new fusion files and database/add_candidate_segment.

7. user (2026-06-13T19:14:05.158Z)

```
            for cw in all_candidate_windows:
                logger.info(f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                            f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                            f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

        # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
        candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
        if store_candidates:
            for i, cw in enumerate(all_candidate_windows):
```

```
                stats = {
                    "peak_velocity": cw["peak_velocity"],
                    "mean_velocity": cw["mean_velocity"],
                    "active_frame_count": cw["active_frame_count"],
                    "track_id_count": cw["track_id_count"],
                }
                candidate_ids[i] = self.db.add_candidate_segment(
                    video_id, detector=self.name,
                    start_time=cw["start"], end_time=cw["end"],
                    chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
                    stats=stats, detection_params=detection_params,
                )

        if not all_candidate_windows:
            return []

        # --- Phase 3: AI provider handoff over padded candidate clips ---
        handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
        os.makedirs(handoff_dir, exist_ok=True)
        logger.info("Phase 3: %s validation on %d candidate windows...",
                    provider_name, len(all_candidate_windows))

        def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
            w_start = max(0, window["start"] - handoff_padding)
            w_end = min(video_duration, window["end"] + handoff_padding)
            w_dur = w_end - w_start
            w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
            if not extract_video_chunk(video_path, w_start, w_dur, w_path, reencode=True):
                return []
            prompt = sport_profile.get_prompt(chunk_duration=w_dur)
            segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
                                                 label=f"Handoff {wi+1}")
            valid = []
            for seg in segments:
                try:
                    seg["start_time"] = float(seg["start_time"]) + w_start
                    seg["end_time"] = float(seg["end_time"]) + w_start
                    seg["_candidate_id"] = candidate_ids[wi]
                    valid.append(seg)
                except (ValueError, TypeError, KeyError) as e:
                    logger.warning("Dropping segment with unparseable timestamps: %s ? %s", seg,
e)
            try:
                os.remove(w_path)
            except OSError:
                pass
            return valid

        ai_segments: List[dict] = []
        with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
            futures = [executor.submit(process_candidate_window, wi, cw)
                       for wi, cw in enumerate(all_candidate_windows)]
            for future in futures:
                ai_segments.extend(future.result())
        try:
            os.rmdir(handoff_dir)
        except OSError:
            pass
```

8. assistant (2026-06-13T19:14:13.556Z)

HEAD confirms claims (1) and (2). The old inline `stats` dict had exactly the same 4 keys in the same order, and the old handoff used `enumerate(all_candidate_windows)`. Now let me read the new fusion files and `add_candidate_segment`.

9. user (2026-06-13T19:14:13.715Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13        never imports torch / touches the GPU.
14      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17        **Persisted candidates remain the full superset** regardless, so the offline
experiment
18        harness can re-score any variant.
19
20      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
23      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24      experiment harness (EXPERIMENT_HARNESS.md), never silently.
25      """
26      from typing import Any, Dict, List, Optional, Tuple
27
28      from backend.pipeline.segmenters.base import segmenter_registry
29      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30      from backend.pipeline.detectors.base import DetectorRunner
31      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33      from backend.pipeline.detectors import rally_gate
34      from backend.pipeline.detectors import fusion_features as ff
35
36
37      class FusionSegmenter(TrajectoryHybridSegmenter):
38          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
39
40          family = "fusion"
41          display_label = "Fusion"
42
43          def __init__(self, db: Any, config: Any):
44              super().__init__(db, config)
45              # Built lazily only if the person cue is enabled (keeps torch/GPU off the
default path).
46              self._person: Optional[Any] = None
47
48          @property
49          def name(self) -> str:
50              return "fusion_hybrid"
51
52          @property
53          def description(self) -> str:
54              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
```

```
rallies, "
                    "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
                    "AI provider sets semantic boundaries.")

    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
        substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
        if substrate == "tracknet":
            cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
            return TrackNetRunner(cfg), {
                "weights_path": cfg.weights_path,
                "queue_length": cfg.queue_length,
                "fusion_substrate": "tracknet",
            }
        wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
        return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}

    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                frame_width: float, video_path: str,
                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
        stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
        # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
        # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
        # over the whole trajectory by rally_gate (camera-agnostic).
        gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
        stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0

        fcfg = idx_cfg.get("fusion", {})
        if fcfg.get("cue_flags", {}).get("person", False):
            stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
        return stats

    def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
                         frame_width: float, video_path: str,
                         fcfg: Dict[str, Any]) -> Dict[str, float]:
        """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
        frame range and compute the scale/translation-invariant person features. Lazily
builds
        ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
        # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
        from backend.pipeline.detectors.person_detector import PersonDetector
        if self._person is None:
            self._person = PersonDetector(self.config)

        frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
        start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
        det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
        fw = det["frame_width"] or frame_width
        fh = det["frame_height"] or 0.0

        poly = fcfg.get("court_polygon")
        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
```

```
107                                             court_polygon=polygon, net=net)
108
109         def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                                 window_stats: List[Dict[str, Any]],
111                                 idx_cfg: Dict[str, Any]) -> List[int]:
112             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113             (0) keep every window ? identical to the substrate. Persisted candidates are
114             untouched (full superset)."""
115             fcfg = idx_cfg.get("fusion", {})
116             min_crossings = int(fcfg.get("min_crossings", 0))
117             min_players = int(fcfg.get("min_players", 0))
118             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120             keep: List[int] = []
121             for i, st in enumerate(window_stats):
122                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                     continue
124                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                     continue
126                 keep.append(i)
127             return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131
```

10. assistant (2026-06-13T19:14:21.935Z)

Now let me verify claim (5) more carefully ? the finding says `window_stats` is now computed unconditionally and the base hook is "pure". But note that the `FusionSegmenter._enrich_candidate_stats` calls `rally_gate.gate_windows` which does real compute. The finding's claim (5) is specifically about the TWINS (wasb_hybrid/tracknet_hybrid using the BASE hook), so that's correct scope. Let me verify the base hook purity and check `add_candidate_segment` for the JSON serialization claim.

11. user (2026-06-13T19:14:22.514Z)

```
240:    def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
241-                              chunk_index: Optional[int] = None, confidence: Optional[float]
= None,
242-                              stats: Optional[Dict[str, Any]] = None,
243-                              detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
244-         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
245-         detector-specific dicts stored as JSON. Returns the new candidate id, or None."""
246-         try:
247-             with self._connect() as conn:
248-                 cursor = conn.cursor()
249-                 cursor.execute(
250-                     """INSERT INTO candidate_segments
251-                        (video_id, detector, chunk_index, start_time, end_time, duration,
252-                         confidence, stats, detection_params)
253-                        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
254-                     (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
255-                      confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
256-                 )
257-                 conn.commit()
258-                 return cursor.lastrowid
259-         except Exception as e:
260-             logger.error(f"Failed to add candidate segment: {e}")
```

```
261-            return None
262-
263-    def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
264-        """Record that a candidate detection was confirmed as a given play_segment."""
265-        try:
266-            with self._connect() as conn:
267-                conn.cursor().execute(
268-                    "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
269-                    (segment_id, candidate_id)
270-                )
271-                conn.commit()
272-            return True
273-        except Exception as e:
274-            logger.error(f"Failed to link candidate {candidate_id} to segment {segment_id}:
{e}")
275-            return False
276-
277-    def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
278-                                 only_unlabeled: bool = False) -> List[Dict[str, Any]]:
279-        """Fetch candidate detections for the annotation / fine-tuning workflow.
280-        `stats` and `detection_params` are deserialized from JSON."""
```

12. assistant (2026-06-13T19:14:26.838Z)

Claim (1) confirmed: `add_candidate_segment` serializes via `json.dumps(stats or {})` at database.py:255. The `stats=window_stats[i]` is passed unchanged at line 216. Since the base `_enrich_candidate_stats` returns the same 4 keys in the same insertion order, the serialized JSON is byte-identical.

Now let me verify the claim that the relevant tests exist and pass. Let me find the test files mentioned.

13. user (2026-06-13T19:14:28.366Z)

```
./tests/test_fusion_hybrid.py
./tests/test_trajectory_hybrid_equivalence.py
```

14. user (2026-06-13T19:14:36.393Z)

```
1       """Equivalence contract for the trajectory-hybrid twins.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins that drifted (the
4       shot_count parse); both are now thin subclasses of TrajectoryHybridSegmenter.
5       These tests run BOTH through the same scenarios and assert identical P3/P4
6       behavior, so any future divergence has to be introduced deliberately, in the
7       base, for both at once.
8       """
9       from unittest.mock import MagicMock, patch
10
11      import pytest
12
13      from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
14      from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
15      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
16
17      TWINS = [
18          (TrackNetHybridSegmenter, "tracknet", "tracknet_hybrid"),
19          (WasbHybridSegmenter, "wasb", "wasb_hybrid"),
20      ]
21
22
23      def _window(start=1.0, end=4.0):
24          return {
25              "start": start, "end": end, "active_frame_count": 30,
```

```
            "peak_velocity": 0.4, "mean_velocity": 0.2, "track_id_count": 1,
            "chunk_index": 0,
        }


    def _runner(windows=None):
        runner = MagicMock()
        runner.healthcheck.return_value = (True, "env ok")
        runner.run_predict.return_value = "out.csv"
        runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
        runner.trajectory_to_action_windows.return_value = windows if windows is not None
else [_window()]
        return runner


    def _run(cls, provider_segments, db=None):
        """Run one segmenter class through the standard mocked pipeline."""
        provider = MagicMock()
        provider.analyze_video.return_value = (provider_segments, {})
        db = db or MagicMock()
        db.add_candidate_segment.return_value = 55
        db.add_play_segment.return_value = 200

        with patch.object(cls, "_build_runner", return_value=(_runner(), {"weights_path":
"w"})), \
             patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), \
             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
             patch("os.environ.get", return_value="dummy_key"):
            preg.get.return_value = provider
            seg = cls(db, {"indexing": {}})
            return db, seg.process_video("v1", "clip.mp4")


    @pytest.mark.parametrize("cls,family,detector", TWINS)
    def test_detector_identity_flows_into_db_rows(cls, family, detector):
        db, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count":
4}])
        assert len(res) == 1
        _, kwargs = db.add_candidate_segment.call_args
        assert kwargs["detector"] == detector
        assert kwargs["detection_params"]["detector"] == detector
        # P4 metadata tags the family + model.
        assert res[0]["metadata"]["detector"].startswith(f"{family}-hybrid-")
        db.link_candidate_to_segment.assert_called_once_with(55, 200)


    @pytest.mark.parametrize("cls,family,detector", TWINS)
    @pytest.mark.parametrize("exchange,expected", [
        (4, 4),            # provider-reported count wins
        ("7", 7),          # numeric strings coerce
        (None, 3),         # explicit null -> duration estimate (2.5s/0.8 -> 3 floor)
        ("garbage", 3),    # unparseable -> same estimate
    ])
    def test_shot_count_behavior_is_identical(cls, family, detector, exchange, expected):
        """The historical drift point: wasb_hybrid relied on int(None) raising into
        the fallback while tracknet_hybrid short-circuited. One implementation now."""
        seg_dict = {"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": exchange}
        _, res = _run(cls, [seg_dict])
        assert res[0]["metadata"]["shot_count"] == expected

```

```
85
86      @pytest.mark.parametrize("cls,family,detector", TWINS)
87      def test_provider_offsets_applied_identically(cls, family, detector):
88          # Window [1,4] + 2.0s default padding -> clip starts at max(0, -1) = 0.0;
89          # provider-relative 0.5 stays 0.5 absolute here.
90          _, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
91          assert res[0]["start_time"] == 0.5
92          assert res[0]["end_time"] == 3.0
93
94
95      def test_base_is_not_registered():
96          from backend.pipeline.segmenters.base import segmenter_registry
97          available = segmenter_registry.list_available()
98          assert "tracknet_hybrid" in available
99          assert "wasb_hybrid" in available
100         # The base engine itself must not appear as a selectable segmenter.
101         assert not any(isinstance(cls, type) and cls is TrajectoryHybridSegmenter
102                        for cls in available.values())
103
104
105     @pytest.mark.parametrize("cls,family,detector", TWINS)
106     def test_persisted_stats_are_exactly_the_four_motion_keys(cls, family, detector):
107         """The fusion-P2 enrichment seam must leave the twins' persisted stats byte-
identical:
108         the base _enrich_candidate_stats is identity, so NO net_crossings / person keys leak
in."""
109         db, _res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
110         _, kwargs = db.add_candidate_segment.call_args
111         assert set(kwargs["stats"]) == {
112             "peak_velocity", "mean_velocity", "active_frame_count", "track_id_count"}
113
114
115     @pytest.mark.parametrize("cls,family,detector", TWINS)
116     def test_base_fusion_hooks_are_identity(cls, family, detector):
117         """Directly pin the two fusion seams on the base: stats = the 4 motion keys
verbatim,
118         and every window is selected for handoff (no gating)."""
119         seg = cls(MagicMock(), {"indexing": {}})
120         cw = _window()
121         stats = seg._enrich_candidate_stats(cw, [], 30.0, 1280.0, "v.mp4", {})
122         assert stats == {"peak_velocity": 0.4, "mean_velocity": 0.2,
123                          "active_frame_count": 30, "track_id_count": 1}
124         assert seg._select_for_handoff([cw, cw, cw], [stats, stats, stats], {}) == [0, 1, 2]
125
```

15. assistant (2026-06-13T19:14:41.292Z)

These tests directly pin claims (1), (2), and (5). Note that `test_persisted_stats_are_exactly_the_four_motion_keys` uses `set(kwargs["stats"])` which only checks key set, not ordering. But `test_base_fusion_hooks_are_identity` asserts `stats == {...}` (full dict equality) and the code returns the literal dict in the same order as HEAD, so insertion order is preserved.

Now let me verify the fusion test file for claims about default behavior, and run the suite.

16. user (2026-06-13T19:14:41.997Z)

```
1       """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3       Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4       keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5       substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6       persistence keeps the superset; and the person cue, when enabled, adds the person
features.
```

```
7       """
8       from types import SimpleNamespace
9       from unittest.mock import MagicMock, patch
10
11      from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12      from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14      # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
15      # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16      WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17               "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18      WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19               "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
20
21
22      def _points():
23          pts = []
24          for f in range(30, 120):       # window A frames: y flips 100<->900 around the net
(~500)
25              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
else 900.0))
26          for f in range(300, 360):      # window B frames: y == net level -> deadband -> no
crossings
27              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
28          return pts
29
30
31      def _runner():
32          r = MagicMock()
33          r.healthcheck.return_value = (True, "env ok")
34          r.run_predict.return_value = "out.csv"
35          r.parse_trajectory_csv.return_value = _points()
36          r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
37          return r
38
39
40      def _run(indexing=None, provider_segments=None):
41          provider = MagicMock()
42          provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
"end_time": 2.0}], {})
43          db = MagicMock()
44          db.add_candidate_segment.side_effect = [101, 102, 103, 104]
45          db.add_play_segment.return_value = 200
46
47          with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
{"weights_path": "w"})), \
48               patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)),
\
49               patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
50               patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
51               patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
52               patch("os.environ.get", return_value="dummy_key"):
53              preg.get.return_value = provider
54              seg = FusionSegmenter(db, {"indexing": indexing or {}})
55              res = seg.process_video("v1", "clip.mp4")
56              return db, provider, res
57
58
59      def _stats_calls(db):
60          """Persisted stats dicts, in call order."""
61          return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]
```

```
62
63
64      def test_net_crossings_persisted_and_person_off_by_default():
65          db, _provider, _res = _run()
66          stats = _stats_calls(db)
67          assert len(stats) == 2                              # both windows persisted
(superset)
68          assert all("net_crossings" in s for s in stats)
69          # Window A (rally) crosses many times; window B (held) zero.
70          assert stats[0]["net_crossings"] >= 2
71          assert stats[1]["net_crossings"] == 0.0
72          # Person cue OFF by default -> NONE of the person features present.
73          assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
74          # The 4 base motion keys are still there.
75          assert all({"peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count"} <= set(s)
76                     for s in stats)
77
78
79      def test_persisted_candidates_are_superset_of_substrate_windows():
80          db, _provider, _res = _run()
81          spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
db.add_candidate_segment.call_args_list]
82          assert spans == [(1.0, 4.0), (10.0, 12.0)]          # exactly the substrate windows,
unfiltered
83
84
85      def test_default_hands_off_all_windows():
86          _db, provider, _res = _run()                        # min_crossings default 0 -> no
gating
87          assert provider.analyze_video.call_count == 2
88
89
90      def test_gate_a_drops_low_crossing_window_from_handoff_only():
91          db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
92          # Persistence is still the full superset (offline substrate intact)...
93          assert len(_stats_calls(db)) == 2
94          # ...but only the high-crossing window A reaches the AI handoff.
95          assert provider.analyze_video.call_count == 1
96
97
98      def test_person_cue_adds_features_when_enabled():
99          fake_pd = MagicMock()
100         fake_pd.detections_per_frame.return_value = {
101             "frames": {30: [(1, (10, 10, 30, 40))], 31: [(1, (12, 10, 32, 40))]},
102             "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
103         }
104         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
return_value=fake_pd):
105             db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
106         stats = _stats_calls(db)
107         assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
108         assert all("net_crossings" in s for s in stats)
109         fake_pd.detections_per_frame.assert_called()         # the person detector WAS used
110
```

17. assistant (2026-06-13T19:14:45.412Z)

Now let me run the test suites to confirm the "all 42 tests pass" claim. Let me check the environment first (CLAUDE.md notes this container lacks numpy/cv2). But the finding claims to have run the suite. Let me check what's importable and run the relevant test files.

18. user (2026-06-13T19:14:55.440Z)

cv2 ok

```
---
......................                                                  [100%]
22 passed in 5.81s
```

19. assistant (2026-06-13T19:15:01.476Z)

cv2 is available here (contrary to the older CLAUDE.md note). 22 tests pass across the two files. The finding claims "42 tests in the 4 relevant files". Let me find the other two test files (fusion_features and rally_gate / person) and run them.

20. user (2026-06-13T19:15:02.641Z)

```
 M tests/test_config.py
 M tests/test_trajectory_hybrid_equivalence.py
?? tests/test_fusion_features.py
?? tests/test_fusion_hybrid.py
===
test_fusion_features.py
test_fusion_hybrid.py
test_person_detector.py
test_rally_gate.py
```

21. assistant (2026-06-13T19:15:05.546Z)

The 4 relevant files are likely: test_trajectory_hybrid_equivalence.py, test_fusion_hybrid.py, test_fusion_features.py, and one of (test_rally_gate / test_person_detector). Let me run all four candidate files to confirm the count and that they pass.

22. user (2026-06-13T19:15:24.403Z)

```
.................................                                       [100%]
36 passed in 6.07s
=== with person_detector ===
.............................                                           [100%]
33 passed in 6.74s
```

23. assistant (2026-06-13T19:15:28.667Z)

The "42 tests in the 4 relevant files" doesn't exactly match my groupings (36 or 33). Let me check the precise counts to find which 4 files sum to 42.

24. user (2026-06-13T19:15:53.238Z)

```
test_trajectory_hybrid_equivalence: 17
test_fusion_hybrid: 5
test_fusion_features: 9
test_rally_gate: 5
test_person_detector: 2
```

25. assistant (2026-06-13T19:16:00.841Z)

The exact "42" count: equivalence(17) + fusion_hybrid(5) + fusion_features(9) + rally_gate(5) + person_detector(2) = 38 across all five. The combination summing to 42 isn't exact among these ? but this is a minor discrepancy in the finding's bookkeeping (the "4 relevant files" / "42 tests" count). The substantive claims (1)-(5) are what matter for the NO-REGRESSION verification, and the test files that pin them all pass.

The exact test count being slightly off (38 vs claimed 42, depending on which 4 files) does not undermine the core finding, which is `severity: info` and a NO-REGRESSION confirmation. The finding may have counted a different set or run before/after a small change. This is not load-bearing.

Now let me verify the remaining substantive claims directly against the code. Let me confirm claim (4) ? the empty-windows guard ordering, and claim (3) ? Phase 3/4 untouched. I already

read both HEAD and current. Let me verify the diff for the other edited files isn't introducing twin regressions, and double-check `rally_gate.gate_windows` exists with the signature used.

26. user (2026-06-13T19:16:02.419Z)

```
91:def gate_windows(points, windows: List[Interval], fps: float,
92-                 min_crossings: int = 2, deadband_frac: float = 0.06) -> List[GatedWindow]:
93-    """Annotate each (start,end) window with its shuttle net-crossing count and a
94-    keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95-    axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96-    axis, net, span = estimate_play_axis(points)
97-    deadband = deadband_frac * span
98-    out: List[GatedWindow] = []
99-    for (s, e) in windows:
100-        wp = _points_in(points, fps, s, e)
101-        c = count_net_crossings(wp, axis, net, deadband)
102-        vis = sum(1 for p in wp if p.visible)
103-        out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104-                               kept=c >= min_crossings))
105-    return out
106-
=== person_detector diff stat ===
 backend/config/models.py                      | 34 ++++++++++++++++++++
 backend/pipeline/detectors/person_detector.py | 47 ++++++++++++++++++++++++++++++++++++++++++++++++
 backend/pipeline/segmenters/__init__.py       |  2 ++
 3 files changed, 83 insertions(+)
```

27. assistant (2026-06-13T19:16:10.381Z)

The edited files only have additions (no deletions to person_detector.py / models.py / __init__.py), confirming the twins' substrate isn't perturbed. `gate_windows` signature matches the call (`gate_windows(points, [(start,end)], fps, min_crossings=0)`), returning `GatedWindow` with `.crossings`.

Now let me verify the `__init__.py` edit just registers fusion (doesn't change twin registration) and confirm the empty-windows guard ordering claim (4) one more time against the current file ? already read at lines 219-220 it sits after the persistence loop and before `_select_for_handoff` at line 225. Confirmed.

Let me check the `__init__.py` diff to be thorough.

28. user (2026-06-13T19:16:11.231Z)

```
diff --git a/backend/pipeline/segmenters/__init__.py b/backend/pipeline/segmenters/__init__.py
index c003d1a..fc5de77 100644
--- a/backend/pipeline/segmenters/__init__.py
+++ b/backend/pipeline/segmenters/__init__.py
@@ -4,6 +4,7 @@ from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
 from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
 from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
 from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
+from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter

 __all__ = [
     "segmenter_registry",
@@ -13,4 +14,5 @@ __all__ = [
     "HybridYoloSegmenter",
     "TrackNetHybridSegmenter",
     "WasbHybridSegmenter",
+    "FusionSegmenter",
 ]
```

29. assistant (2026-06-13T19:16:28.396Z)

Pure additive ? the twin imports/registrations are untouched. This confirms the twins are not perturbed at the registration layer.

I have now adversarially verified every substantive claim. Let me consolidate my findings.

Verification summary against the actual code:

- **Claim (1) PERSISTED STATS UNCHANGED ? HOLDS.** Base `_enrich_candidate_stats` (trajectory_hybrid.py:89-94) returns the 4 motion keys in the exact same insertion order as the old inline `stats` dict (HEAD:179-184, verified via `git show`). `add_candidate_segment` serializes via `json.dumps(stats or {})` (database.py:255), `stats=window_stats[i]` passed unchanged (line 216). Pinned by `test_persisted_stats_are_exactly_the_four_motion_keys` AND `test_base_fusion_hooks_are_identity` (full-dict equality, so order is pinned too).

- **Claim (2) HANDOFF SENDS ALL WINDOWS ? HOLDS.** Base `_select_for_handoff` returns `list(range(len(windows)))` (line 104). New futures comprehension `for wi in handoff_indices` (lines 258-259) over `[0..N-1]` produces the same `(wi, all_candidate_windows[wi])` pairs and same result-collection order as the old `enumerate(all_candidate_windows)` (HEAD). `candidate_ids[wi]` linkage unaffected. Pinned by `test_base_fusion_hooks_are_identity`.

- **Claim (3) PHASE-3/4 UNCHANGED ? HOLDS.** The full diff shows `process_candidate_window` body, padding math, provider call, timestamp offsetting, `os.remove`/`os.rmdir`, and the entire Phase-4 loop are untouched.

- **Claim (4) EMPTY-WINDOWS GUARD INTACT ? HOLDS.** `if not all_candidate_windows: return []` (lines 219-220) sits after the persistence loop and before `_select_for_handoff` (line 225). On empty input the persistence loop runs zero times and the function returns before any gating.

- **Claim (5) store_candidates=False NO BEHAVIOURAL CHANGE for twins ? HOLDS.** `window_stats` is computed unconditionally (lines 203-206), but the BASE hook is pure (reads dict keys only). Dead compute, no observable effect for the twins.

- **Default FusionSegmenter parity ? HOLDS.** `_build_runner` reads `WasbConfig.from_indexing_cfg(idx_cfg)` (fusion_hybrid.py:67-68); default `min_crossings=0`/`min_players=0`/person OFF makes `_select_for_handoff` keep every index (lines 116-127). Pinned by `test_default_hands_off_all_windows` + `test_persisted_candidates_are_superset_of_substrate_windows`.

One minor bookkeeping discrepancy (not load-bearing): the finding says "All 42 tests in the 4 relevant files pass." Actual collected counts are: equivalence=17, fusion_hybrid=5, fusion_features=9, rally_gate=5, person_detector=2. No grouping of 4 of these files sums to exactly 42 (e.g., 17+5+9+5=36, or 17+5+9+2=33). All tests I ran pass (38 across all five files). This is a cosmetic miscount in the review note, not an error in the code or the substantive verification.

The finding is `severity: info` and is a NO-REGRESSION confirmation. All five technical claims hold against the actual code; only the test-count tally is slightly off. The finding is real and correct in substance.

30. user (2026-06-13T19:16:41.039Z)

Structured output provided successfully

31. assistant (2026-06-13T19:16:49.289Z)

I've completed the adversarial verification. The finding is **real** (severity: info ? a NO-REGRESSION confirmation).

All five substantive claims hold against the actual code:
- **(1)** Base `_enrich_candidate_stats` returns the 4 motion keys in the exact same insertion order as HEAD's inline dict (verified via `git show`), serialized identically by `add_candidate_segment` (database.py:255).
- **(2)** Base `_select_for_handoff` = `list(range(N))` produces the same futures pairs/order as the old `enumerate`.
- **(3)** Phase-3/4 bodies untouched per the full diff.

- **(4)** Empty-windows guard still sits between persistence and gating (lines 219-225).
- **(5)** Unconditional `window_stats` compute is dead compute for the twins (pure base hook).
- Default FusionSegmenter parity verified (WASB substrate, thresholds 0, person OFF).

The edited non-twin files (`__init__.py`, `person_detector.py`, `models.py`) are purely additive (0 deletions), so the twin substrate is untouched. The referenced test files all pass.

The only inaccuracy is cosmetic and non-load-bearing: the note's "42 tests in the 4 relevant files" doesn't match any 4-file subset of the five fusion-related test files (actual counts: 17+5+9+5+2 = 38 across all five, all green). That's a tally typo in the review note, not a code or verification defect ? it doesn't undermine the finding.